Software Engineering notes by K. Adisesha

Analysis ad design of algorithms (Bengaluru Central University)

Software Engineering

Software and software systems are everywhere. The economies of ALL developed nations are dependent on software. More and more systems are software controlled. Software engineering is concerned with theories, methods and tools for professional software development. Expenditure on software represents a significant fraction of GNP in all developed countries.

Applying software engineering principles helps address the two main factors of software failures:

- **Increasing demands:** As new software engineering techniques help us to build larger, more complex systems, the demands change. Systems have to be built and delivered more quickly; larger, even more complex systems are required; systems have to have new capabilities that were previously thought to be impossible.
- **Low expectations:** It is relatively easy to write computer programs without using software engineering methods and techniques. Computer programming is NOT software engineering.

FAQ about software engineering

What is software?

Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.

What are the attributes of good software?

Good software should deliver the required functionality and performance to the user and should be maintainable, dependable, and usable.

What is software engineering?

Software engineering is an engineering discipline that is concerned with all aspects of software production.

What are the fundamental software engineering activities?

Software specification, software development, software validation and software evolution.

What is the difference between software engineering and computer science?

Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.

What is the difference between software engineering and system engineering?

System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this more general process.

What are the key challenges facing software engineering?

Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.

What are the costs of software engineering?

Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.

What are the best software engineering techniques and methods?

While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another.

What differences has the web made to software engineering?

The web has led to the availability of software services and the possibility of developing highly distributed service-based systems. Web-based systems development has led to important advances in programming languages and software reuse.

Essential attributes of good software

- **Maintainability:** Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
- **Dependability and security:** Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
- **Efficiency:** Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
- **Acceptability:** Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

Software engineering: a definition

Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use. It is an engineering discipline because it uses appropriate theories and methods to solve problems bearing in mind organizational and financial constraints. Software engineering focuses on all aspects of software production and not just on the technical process of development; it includes project management and the development of tools, methods etc. to support software production.

It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project. For most types of system, the majority of costs are the costs of changing the software after it has gone into use.

Any software process includes four types of activities:

- Software **specification**, where customers and engineers define the software that is to be produced and the constraints on its operation.
- Software **development**, where the software is designed and programmed.
- Software **validation**, where the software is checked to ensure that it is what the customer requires.
- Software **evolution**, where the software is modified to reflect changing customer and market requirements.

General issues affecting most software

- **Heterogeneity:** Increasingly, systems are required to operate as distributed systems across networks that include different types of computer and mobile devices.

- **Business and social change:** Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software.

- **Security and trust:** As software is intertwined with all aspects of our lives, it is essential that we can trust that software.

Software engineering fundamentals

These software engineering fundamentals that apply to all types of software system:

Software process: Systems should be developed using a managed and understood development process. The organization developing the software should plan the development process and have clear ideas of what will be produced and when it will be completed. Of course, different processes are used for different types of software.

Focus on reliability: Dependability and performance are important for all types of systems. Software should behave as expected, without failures and should be available for use when it is required. It should be safe in its operation and, as far as possible, should be secure against external attack. The system should perform efficiently and should not waste resources.

Importance of requirements: Understanding and managing the software specification and requirements (what the software should do) are important. You have to know what different customers and users of the system expect from it and you have to manage their expectations so that a useful system can be delivered within budget and to schedule.

Leverage software reuse: You should make as effective use as possible of existing resources. This means that, where appropriate, you should reuse software that has already been developed rather than write new software.

Software engineering ethics: Some issues of professional responsibility:

Confidentiality: You should normally respect the confidentiality of your employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

Competence: You should not misrepresent your level of competence. You should not knowingly accept work that is outside your competence.

Intellectual property rights: You should be aware of local laws governing the use of intellectual property such as patents and copyright. You should be careful to ensure that the intellectual property of employers and clients is protected.

Computer misuse: You should not use your technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses or other malware).

Software Processes

A software process is a structured set of activities required to develop a software system. Note that we are talking about a "software process" -- not a "software *development* process."

There are many different kinds of software processes, but each and every one of them involve these four types of fundamental activities:

- Software **specification** - defining what the system should do;
- Software **design and implementation** - defining the organization of the system and implementing the system;
- Software **validation** - checking that it does what the customer wants;
- Software **evolution** - changing the system in response to changing customer needs.

A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective. When we describe and discuss software processes, we usually talk about the activities in these processes such as specifying a data model, designing a user interface, etc. and the ordering of these activities. Process descriptions may also include:

- **Products (what)**, which are the outcomes of a process activity;
- **Roles (who)**, which reflect the responsibilities of the people involved in the process;
- Pre- and post-**conditions (how)**, which are statements that are true before and after a process activity has been enacted or a product produced.

Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan. In **agile** processes, planning is incremental and it is easier to change the process to reflect changing customer requirements. In practice, most practical processes include elements of both plan-driven and agile approaches.

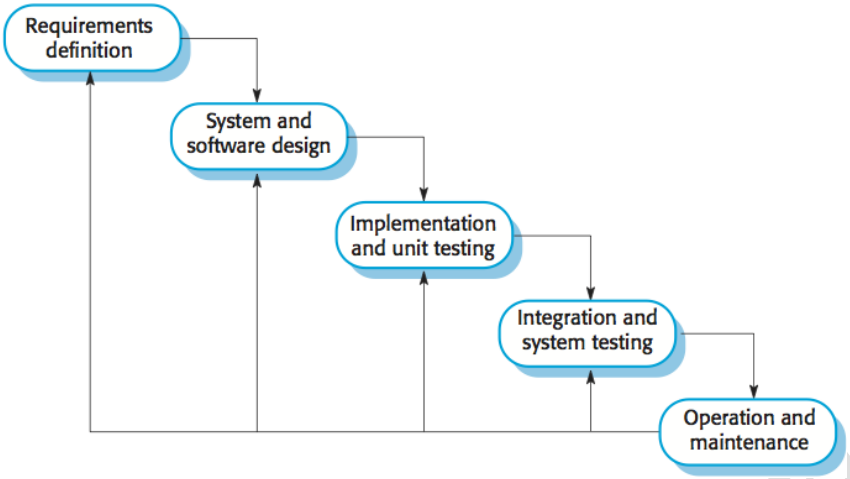
Software process models

The waterfall model: Plan-driven model. Separate and distinct phases of specification, software design, implementation, testing, and maintenance.

Incremental development: Specification, development and validation are interleaved. The system is developed as a series of versions (increments), with each version adding functionality to the previous version. May be plan-driven or agile.

Integration and configuration: Based on the existence of a significant number of reusable components/systems. The system development process focuses on integrating these components into a system rather than developing them from scratch. May be plan-driven or agile.
In practice, most large systems are developed using a process that incorporates elements from all of these models.

The waterfall model:



There are separate identified **phases** in the waterfall model:

Requirements analysis and definition: The system's services, constraints, and goals are established by consultation with system users. They are then defined in detail and serve as a system specification.

System and software design: The systems design process allocates the requirements to either hardware or software systems by establishing an overall system architecture. Software design involves identifying and describing the fundamental software system abstractions and their relationships.

Implementation and unit testing: During this stage, the software design is realized as a set of programs or program units. Unit testing involves verifying that each unit meets its specification.

Integration and system testing: The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met. After testing, the software system is delivered to the customer.

Operation and maintenance: Normally (although not necessarily), this is the longest life cycle phase. The system is installed and put into practical use. Maintenance involves correcting errors which were not discovered in earlier stages of the life cycle, improving the implementation of system units and enhancing the system's services as new requirements are discovered.

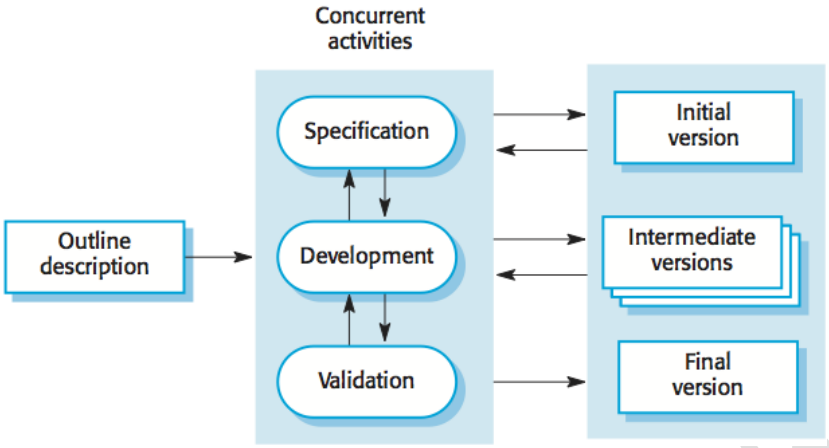
The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway. In principle, a phase has to be complete before moving onto the next phase.

Waterfall model **problems** include:

- **Difficult to address change:** Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements. Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process. Few business systems have stable requirements.

- **Very few real-world applications:** The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites. In those circumstances, the plan-driven nature of the waterfall model helps coordinate the work.

Incremental development model



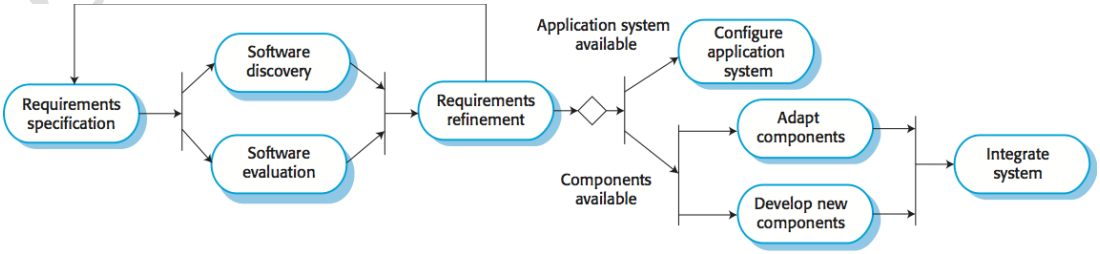
Benefits of incremental development:

- **Lower cost of changes:** The cost of accommodating changing customer requirements is reduced. The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
- **Frequent feedback:** It is easier to get customer feedback on the development work that has been done. Customers can comment on demonstrations of the software and see how much has been implemented.
- **Faster delivery:** More rapid delivery and deployment of useful software to the customer is possible. Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Problems with incremental development (from the management perspective):

- **The process is not visible:** Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
- **System structure tends to degrade as new increments are added:** Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

Integration and configuration



This approach is based on systematic reuse where systems are integrated from existing components or COTS (Commercial-off-the-shelf) systems. Process stages include:

- Component analysis;
- Requirements modification;
- System design with reuse;
- Development and integration.

Reuse is now the standard approach for building many types of business system.

Types of software components:

- **Web services** that are developed according to service standards and which are available for remote invocation.
- Collections of objects that are developed as a **package** to be integrated with a component framework such as .NET or J2EE.
- Stand-alone **commercial-off-the-shelf** systems (COTS) that are configured for use in a particular environment.

Software process activities

Real software processes are inter-leaved sequences of technical, collaborative and managerial activities with the overall goal of specifying, designing, implementing and testing a software system.

The four basic process activities of specification, development, validation and evolution are organized differently in different development processes. In the waterfall model, they are organized in sequence, whereas in incremental development they are interleaved.

Software specification

The process of establishing what services are required and the constraints on the system's operation and development.

Requirements engineering process:

- **Feasibility study**: is it technically and financially feasible to build the system?
- Requirements **elicitation and analysis**: what do the system stakeholders require or expect from the system?
- Requirements **specification**: defining the requirements in detail
- Requirements **validation**: checking the validity of the requirements

Software design and implementation

The process of converting the system specification into an executable system.

- **Software design**: design a software structure that realizes the specification;
- **Implementation**: translate this structure into an executable program;

The activities of design and implementation are closely related and may be interleaved.

Design activities include:

- **Architectural design**: identify the overall structure of the system, the principal components (sometimes called sub-systems or modules), their relationships and how they are distributed.
- **Interface design**: define the interfaces between system components.
- **Component design**: take each system component and design how it will operate.

- **Database design**: design the system data structures and how these are to be represented in a database.

Software validation

Verification and validation (V & V) is intended to show that a system conforms to its specification and meets the requirements of the system customer.

- **Validation**: are we building the right product (what the customer wants)?
- **Verification**: are we building the product right?

V & V involves checking and review processes and system testing. System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.

Testing is the most commonly used V & V activity and includes the following stages:

- **Development or component testing**: individual components are tested independently; components may be functions or objects or coherent groupings of these entities.
- **System testing**: testing of the system as a whole, testing of emergent properties is particularly important.
- **Acceptance testing**: testing with customer data to check that the system meets the customer's needs.

Software evolution

Software is inherently flexible and can change. As requirements change through changing business circumstances, the software that supports the business must also evolve and change. Although there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.

Coping with change

Change is inevitable in all large software projects. Business changes lead to new and changed system requirements New technologies open up new possibilities for improving implementations. Changing platforms require application changes. Change leads to rework so the costs of change include both rework (e.g. re-analyzing requirements) as well as the costs of implementing new functionality.

Two strategies to **reduce the costs** of rework:

- **Change avoidance:** The software process includes activities that can anticipate possible changes before significant rework is required. For example, a prototype system may be developed to show some key features of the system to customers.
- **Change tolerance:** The process is designed so that changes can be accommodated at relatively low cost. This normally involves some form of incremental development. Proposed changes may be implemented in increments that have not yet been developed. If this is impossible, then only a single increment (a small part of the system) may have be altered to incorporate the change.

Software prototyping

A prototype is an initial version of a system used to demonstrate concepts and try out design options. A prototype can be used in:

- The requirements engineering process to help with requirements elicitation and validation;
- In design processes to explore options and develop a UI design;
- In the testing process to run back-to-back tests.

Benefits of prototyping:

- Improved system usability.
- A closer match to users' real needs.
- Improved design quality.
- Improved maintainability.
- Reduced development effort.

Prototypes may be based on rapid prototyping languages or tools. They may involve **leaving out functionality**:

- Prototype should focus on areas of the product that are not well-understood;
- Error checking and recovery may not be included in the prototype;
- Focus on functional rather than non-functional requirements such as reliability and security.

Prototypes should be **discarded** after development as they are not a good basis for a production system:

- It may be impossible to tune the system to meet non-functional requirements;
- Prototypes are normally undocumented;
- The prototype structure is usually degraded through rapid change;
- The prototype probably will not meet normal organizational quality standards.

Incremental development/delivery

Rather than deliver the system as a single delivery, the development and delivery is broken down into increments with each increment delivering part of the required functionality. User requirements are prioritized and the highest priority requirements are included in early increments. Once the development of an increment is started, the requirements are frozen though requirements for later increments can continue to evolve.

Advantages of incremental delivery:

- Customer value can be delivered with each increment so system functionality is available earlier.
- Early increments act as a prototype to help elicit requirements for later increments.
- Lower risk of overall project failure.
- The highest priority system services tend to receive the most testing.

Incremental delivery problems:

- Most systems require a set of basic facilities that are used by different parts of the system. As requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities that are needed by all increments.

- The essence of iterative processes is that the specification is developed in conjunction with the software. However, this conflicts with the procurement model of many organizations, where the complete system specification is part of the system development contract.

Process improvement

Many software companies have turned to software process improvement as a way of enhancing the quality of their software, reducing costs or accelerating their development processes. Process improvement means understanding existing processes and changing these processes to increase product quality and/or reduce costs and development time.

- **Process maturity approach:** Focuses on improving process and project management and introducing good software engineering practice. The level of process maturity reflects the extent to which good technical and management practice has been adopted in organizational software development processes.
- **Agile approach:** Focuses on iterative development and the reduction of overheads in the software process. The primary characteristics of agile methods are rapid delivery of functionality and responsiveness to changing customer requirements.

Process improvement activities form a continuous cycle with a feedback loop:

- **Measure** one or more attributes of the software process or product. These measurements forms a baseline that help decide if process improvements have been effective.
- **Analyze** the current process and identify any bottlenecks.
- **Change** the process to address some of the identified process weaknesses. These are introduced and the cycle resumes to collect data about the effectiveness of the changes.

Process measurement

- Wherever possible, quantitative process data should be collected.
- Process measurements should be used to assess process improvements.
- Metrics may include:
  - Time taken for process activities to be completed, e.g. calendar time or effort to complete an activity or process.
  - Resources required for processes or activities, e.g. total effort in person-days.
  - Number of occurrences of a particular event, e.g. number of defects discovered.

The SEI capability maturity model

- **Initial:** Essentially uncontrolled
- **Repeatable:** Product management procedures defined and used
- **Defined:** Process management procedures and strategies defined and used
- **Managed:** Quality management strategies defined and used
- **Optimizing:** Process improvement strategies defined and used

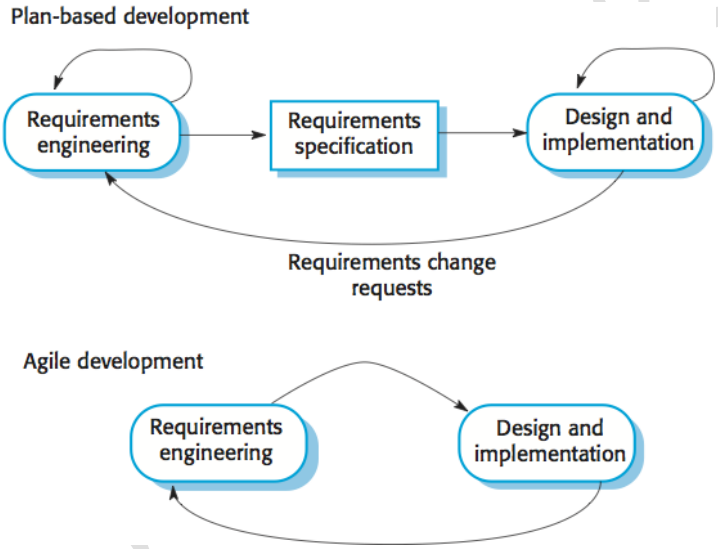
Agile Software Development

Rapid development and delivery is now often the most important requirement for software systems. Businesses operate in a fast-changing requirement and it is practically impossible to produce a set of stable software requirements. Software has to evolve quickly to reflect changing business needs.

Agile development methods emerged in the late 1990s whose aim was to radically reduce the delivery time for working software systems:

- Program specification, design, and implementation are **interleaved**
- The system is developed as a series of frequent versions or **increments**
- **Stakeholders** involved in version specification and evaluation
- Extensive **tool support** (e.g. automated testing tools) used to support development
- **Minimal documentation** - focus on working code

Plan-based vs agile development



Plan-driven development

A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance. Not necessarily waterfall model: plan-driven, incremental development is possible. Iteration occurs within activities.

Agile development

Specification, design, implementation and testing are inter-leaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Most projects include elements of plan-driven and agile processes. Deciding on the balance depends on many technical, human, and organizational issues.

Agile methods

Dissatisfaction with the overheads involved in software design methods of the 1980s and 1990s led to the creation of agile methods. These methods:

- **Focus on the code** rather than the design.
- Are based on an **iterative approach** to software development.
- Are intended to deliver working software quickly and evolve this quickly to **meet changing requirements**.

The aim of agile methods is to reduce overheads in the software process (e.g. by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework.

Manifesto for Agile Software Development:

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- *Individuals and interactions over processes and tools*
- *Working software over comprehensive documentation*
- *Customer collaboration over contract negotiation*
- *Responding to change over following a plan*

That is, while there is value in the items on the right, we value the items on the left more.

The principles of agile methods:

- **Customer involvement:** Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.

- **Incremental delivery:** The software is developed in increments with the customer specifying the requirements to be included in each increment.

- **People not process:** The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.

- **Embrace change:** Expect the system requirements to change and so design the system to accommodate these changes.

- **Maintain simplicity:** Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Agile method **applicability**:

- Product development where a software company is developing a **small or medium-sized product**.

- Custom system development within an organization, where there is a clear **commitment from the customer** to become involved in the development process and where there are not a lot of external rules and regulations that affect the software.
- Because of their focus on small, tightly-integrated teams, there are **problems in scaling** agile methods to large systems.

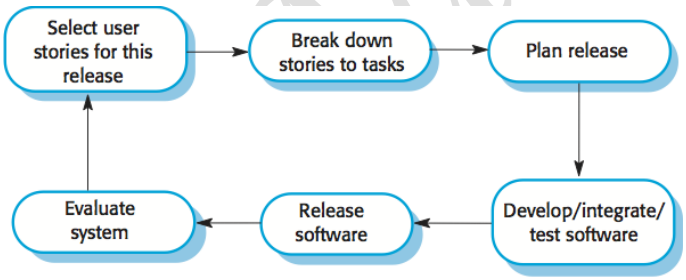
Problems with agile methods:

- It can be difficult to keep the interest of **customers** who are involved in the process.
- **Team members** may be unsuited to the intense involvement that characterizes agile methods.
- Prioritizing changes can be difficult where there are **multiple stakeholders**.
- **Maintaining simplicity** requires extra work.
- **Contracts** may be a problem as with other approaches to iterative development.

Extreme programming

Perhaps the best-known and a very influential agile method, Extreme Programming (XP) takes an 'extreme' approach to iterative development:

- New versions may be built several times per day;
- Increments are delivered to customers every 2 weeks;
- All tests must be run for every build and the build is only accepted if tests run successfully.



This is how XP supports **agile principles**:

- Incremental development is supported through **small, frequent system releases**.
- Customer involvement means **full-time customer engagement** with the team.
- People not process through **pair programming**, **collective ownership**, and a process that avoids long working hours.
- Change supported through **regular system releases**.
- Maintaining simplicity through **constant refactoring** of code.

Influential XP practices

Extreme programming has a technical focus and is not easy to integrate with management practice in most organizations. Consequently, while agile development uses practices from XP, the method as originally defined is not widely used.

Key practices of XP include:

User stories for specification: In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements. User requirements are expressed as user stories or scenarios. These

are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates. The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

Refactoring: Conventional wisdom in software engineering is to design for change. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle. XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated. Rather, it proposes constant code improvement (refactoring) to make changes easier when they have to be implemented. Programming teams look for possible software improvements and make these improvements even where there is no immediate need for them. This improves the understandability of the software and so reduces the need for documentation. Changes are easier to make because the code is well-structured and clear. However, some changes require architecture refactoring and this is much more expensive.

Test-first development: Testing is central to XP and XP has developed an approach where the program is tested after every change has been made.

Test-driven development: writing tests before code clarifies the requirements to be implemented. Tests are written as programs rather than data so that they can be executed automatically. The test includes a check that it has executed correctly (usually relies on a testing framework such as Junit). All previous and new tests are run automatically when new functionality is added, thus checking that the new functionality has not introduced errors.

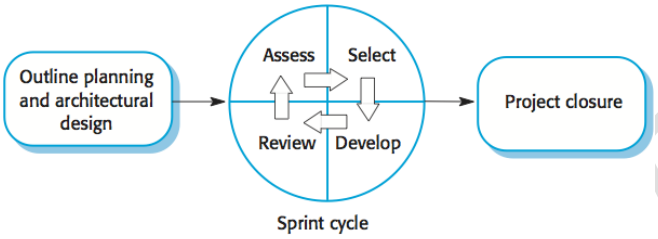
Customer involvement: The role of the customer in the testing process is to help develop acceptance tests for the stories that are to be implemented in the next release of the system. The customer who is part of the team writes tests as development proceeds. All new code is therefore validated to ensure that it is what the customer needs. However, people adopting the customer role have limited time available and so cannot work full-time with the development team. They may feel that providing the requirements was enough of a contribution and so may be reluctant to get involved in the testing process.

Pair programming: Pair programming involves programmers working in pairs, developing code together. This helps develop common ownership of code and spreads knowledge across the team. It serves as an informal review process as each line of code is looked at by more than one person. It encourages refactoring as the whole team can benefit from improving the system code. In pair programming, programmers sit together at the same computer to develop the software. Pairs are created dynamically so that all team members work with each other during the development process. The sharing of knowledge that happens during pair programming is very important as it reduces the overall risks to a project when team members leave. Pair programming is not necessarily inefficient and there is some evidence that suggests that a pair working together is more efficient than two programmers working separately.

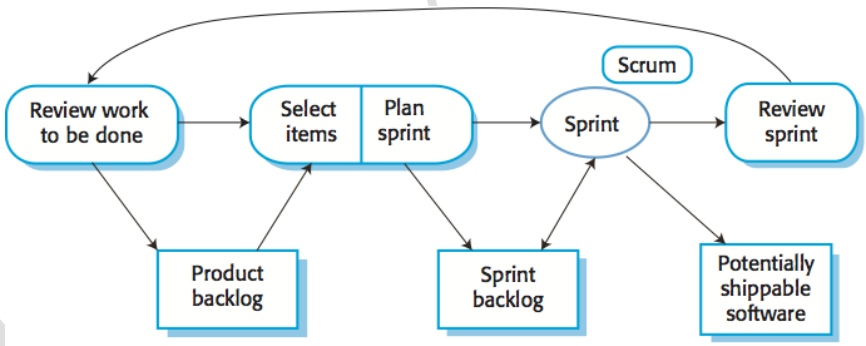
Scrum

The Scrum approach is a general agile method but its focus is on managing iterative development rather than specific agile practices. There are three phases in Scrum:

1. The initial phase is an outline planning phase where you establish the general objectives for the project and design the software architecture.
2. This is followed by a series of **sprint** cycles, where each cycle develops an increment of the system.
3. The project closure phase wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.



Sprints are fixed length, normally 2-4 weeks. They correspond to the development of a release of the system in XP. The starting point for planning is the **product backlog**, which is the list of work to be done on the project. The selection phase involves all of the project team who work with the customer (**product owner**) to select the features and functionality to be developed during the sprint. Once these are agreed, the team organize themselves to develop the software. During this stage the team is relatively isolated from the product owner and the organization, with all communications channelled through the **ScrumMaster**. The role of the ScrumMaster is to protect the development team from external distractions. At the end of the sprint the work done is reviewed and presented to stakeholders (including the product owner). **Velocity** is calculated during the **sprint review**; it provides an estimate of how much product backlog the team can cover in a single sprint. Understanding the team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring and improving performance. The next sprint cycle then begins.



The ScrumMaster is a facilitator who arranges short daily meetings (**daily scrums**), tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with the product owner and management outside of the team. The whole team attends daily scrums where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.

Advantages of scrum include:

- The product is broken down into a set of **manageable and understandable chunks**.
- Unstable requirements do not hold up **progress**.
- The whole team have visibility of everything and consequently **team communication** is improved.
- Customers see **on-time delivery** of increments and gain feedback on how the product works.

- **Trust** between customers and developers is established and a positive culture is created in which everyone expects the project to succeed.

Scaling agile methods

Agile methods have proved to be **successful for small and medium sized projects** that can be developed by a small co-located team. It is sometimes argued that the success of these methods comes because of improved communications which is possible when everyone is working together. Scaling up agile methods involves changing these to cope with larger, longer projects where there are multiple development teams, perhaps working in different locations.

Two perspectives on scaling of agile methods:

'Scaling up': Using agile methods for developing large software systems that cannot be developed by a small team. For large systems development, it is not possible to focus only on the code of the system; you need to do more up-front design and system documentation. Cross-team communication mechanisms have to be designed and used, which should involve regular phone and video conferences between team members and frequent, short electronic meetings where teams update each other on progress. Continuous integration, where the whole system is built every time any developer checks in a change, is practically impossible; however, it is essential to maintain frequent system builds and regular releases of the system.

'Scaling out': How agile methods can be introduced across a large organization with many years of software development experience. Project managers who do not have experience of agile methods may be reluctant to accept the risk of a new approach. Large organizations often have quality procedures and standards that all projects are expected to follow and, because of their bureaucratic nature, these are likely to be incompatible with agile methods. Agile methods seem to work best when team members have a relatively high skill level. However, within large organizations, there are likely to be a wide range of skills and abilities. There may be cultural resistance to agile methods, especially in those organizations that have a long history of using conventional systems engineering processes.

Requirements Engineering

Requirements engineering (RE) is the process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed. The **requirements** themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process. Requirements may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification. As much as possible, requirements should describe **what** the system should do, but **not how** it should do it.

Two kinds of requirements based on the intended purpose and target audience:

- **User requirements:** High-level abstract requirements written as statements, in a natural language plus diagrams, of what services the system is expected to provide to system users and the constraints under which it must operate.
- **System requirements:** Detailed description of what the system should do including the software system's functions, services, and operational constraints. The system requirements document (sometimes called a functional specification) should define exactly what is to be implemented. It may be part of the contract between the system buyer and the software developers.

Three classes of requirements:

- **Functional requirements:** Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations. May state what the system should not do.
- **Non-functional requirements:** Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc. Often apply to the system as a whole rather than individual features or services.
- **Domain requirements:** Constraints on the system derived from the domain of operation.

Functional requirements

Functional requirements describe **functionality** or system services. They depend on the type of software, expected users and the type of system where the software is used.

- Functional user requirements may be **high-level statements of what the system should do**.
- Functional system requirements should describe the system services in detail.

Problems arise when requirements are not precisely stated. Ambiguous requirements may be interpreted in different ways by developers and users. In principle, requirements should be both

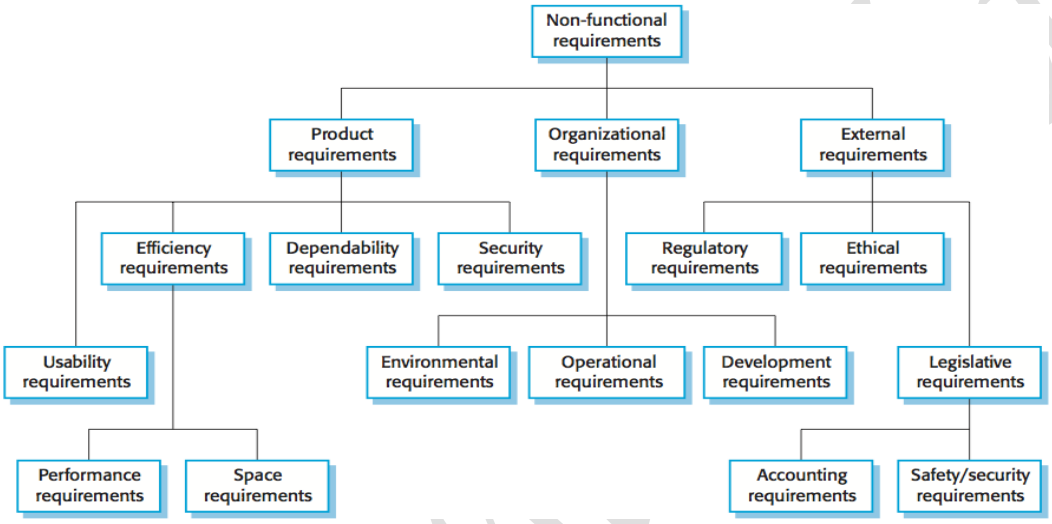
- **Complete**: they should include descriptions of all facilities required, and
- **Consistent**: there should be no conflicts or contradictions in the descriptions of the system facilities.

In practice, it is impossible to produce a complete and consistent requirements document.

Non-functional requirements

Non-functional requirements define system properties and constraints e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc. Process requirements may also be specified mandating a particular IDE, programming language or development method. Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless.

Non-functional requirements may affect the overall architecture of a system rather than the individual components. A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required. It may also generate requirements that restrict existing requirements.



Three classes of non-functional requirements:

- **Product requirements:** Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.
- **Organizational requirements:** Requirements which are a consequence of organizational policies and procedures e.g. process standards used, implementation requirements, etc.
- **External requirements:** Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Non-functional requirements may be very difficult to state precisely and imprecise requirements may be difficult to verify. If they are stated as a **goal** (a general intention of the user such as ease of use), they should be rewritten as a **verifiable** non-functional requirement (a statement using some **quantifiable metric** that can be objectively tested). Goals are helpful to developers as they convey the intentions of the system users.

Domain requirements

The system's operational domain imposes requirements on the system. Domain requirements may be new functional or non-functional requirements, constraints on existing requirements, or define specific computations. If domain requirements are not satisfied, the system may be unworkable. Two main **problems** with domain requirements:

- **Understandability:** Requirements are expressed in the language of the application domain, which is not always understood by software engineers developing the system.
- **Implicitness:** Domain specialists understand the area so well that they do not think of making the domain requirements explicit.

Requirements engineering process

Processes vary widely depending on the application domain, the people involved and the organization developing the requirements. In practice, requirements engineering is an **iterative process**, in which the following generic activities are interleaved:

- Requirements **elicitation**;
- Requirements **analysis**;
- Requirements **validation**;
- Requirements **management**.

Requirements elicitation and analysis

Software engineers work with a range of system **stakeholders** to find out about the application domain, the services that the system should provide, the required system performance, hardware constraints, other systems, etc. Stages include:

- **Requirements discovery:** Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.

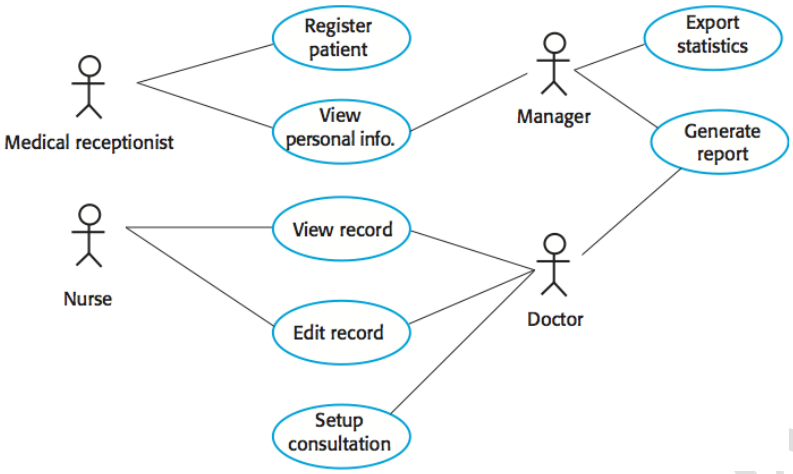
- **Requirements classification and organization:** Groups related requirements and organizes them into coherent clusters.

- **Prioritization and negotiation:** Prioritizing requirements and resolving requirements conflicts.

- **Requirements specification:** Requirements are documented and input into the next round of the spiral.

Closed (based on pre-determined list of questions) and open **interviews** with stakeholders are a part of the RE process. **User stories** and **scenarios** are real-life examples of how a system can be used, which are usually easy for stakeholders to understand. Scenarios should include descriptions of the starting situation, normal flow of events, what can go wrong, other concurrent activities, the state of the system when the scenario finishes.

Use-cases are a scenario-based technique in the UML which identify the actors in an interaction and which describe the interaction itself. A set of use cases should describe all possible interactions with the system.



Problems to look for during requirements elicitation and analysis:

- **Stakeholders don't know what they really want.**
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organizational and political factors may influence the system requirements.
- The requirements change during the analysis process.
- New stakeholders may emerge and the business environment may change.

Requirements specification

Requirements specification is the process of **writing down the user and system requirements** in a requirements document. User requirements have to be understandable by end-users and customers who do not have a technical background. System requirements are more detailed requirements and may include more technical information. The requirements may be part of a contract for the system development and it is important that these are as complete as possible.

In principle, requirements should state what the system should do and the design should describe how it does this. In practice, **requirements and design are inseparable**.

User requirements are almost always written in **natural language** supplemented by appropriate diagrams and tables in the requirements document. System requirements may also be written in natural language but other notations based on forms, graphical system models, or mathematical system models can also be used. Natural language is expressive, intuitive and universal. This means that the requirements can be understood by users and customers.

Structured natural language is a way of writing system requirements where the freedom of the requirements writer is limited and all requirements are written in a standard way. This approach maintains most of the expressiveness and understand-ability of natural language but ensures that some uniformity is imposed on the specification.

Requirements validation

Requirements validation is concerned with demonstrating that the requirements define the system that the customer really wants. Requirements error costs are high so validation is very important.

What **problems** to look for:

- **Validity**: does the system provide the functions which best support the customer's needs?
- **Consistency**: are there any requirements conflicts?
- **Completeness**: are all functions required by the customer included?
- **Realism**: can the requirements be implemented given available budget and technology?
- **Verifiability**: can the requirements be checked?

Requirements validation techniques:

Requirements reviews: Systematic manual analysis of the requirements. Regular reviews should be held while the requirements definition is being formulated. What to look for:

- **Verifiability**: is the requirement realistically testable?
- **Comprehensibility**: is the requirement properly understood?
- **Traceability**: is the origin of the requirement clearly stated?
- **Adaptability**: can the requirement be changed without a large impact on other requirements?

Prototyping: Using an executable model of the system to check requirements.

Test-case generation: Developing tests for requirements to check testability.

Requirements change

Requirements management is the process of managing changing requirements during the requirements engineering process and system development. New requirements emerge as a system is being developed and after it has gone into use. Reasons why requirements change after the system's deployment:

- The business and technical environment of the system always changes after installation.
- The people who pay for a system and the users of that system are rarely the same people.
- Large systems usually have a diverse user community, with many users having different requirements and priorities that may be conflicting or contradictory.

System Modelling

System modelling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system. It is about representing a system using some kind of graphical notation, which is now almost always based on notations in the **Unified Modelling Language** (**UML**). Models help the analyst to understand the functionality of the system; they are used to communicate with customers.

Models can explain the system from **different perspectives**:

- An **external** perspective, where you model the context or environment of the system.
- An **interaction** perspective, where you model the interactions between a system and its environment, or between the components of a system.
- A **structural** perspective, where you model the organization of a system or the structure of the data that is processed by the system.
- A **behavioral** perspective, where you model the dynamic behavior of the system and how it responds to events.

Five types of UML diagrams that are the most useful for system modelling:

- **Activity** diagrams, which show the activities involved in a process or in data processing.
- **Use case** diagrams, which show the interactions between a system and its environment.
- **Sequence** diagrams, which show interactions between actors and the system and between system components.
- **Class** diagrams, which show the object classes in the system and the associations between these classes.
- **State** diagrams, which show how the system reacts to internal and external events.

Models of both new and existing system are used during **requirements engineering**. Models of the **existing systems** help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system. Models of the **new system** are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.

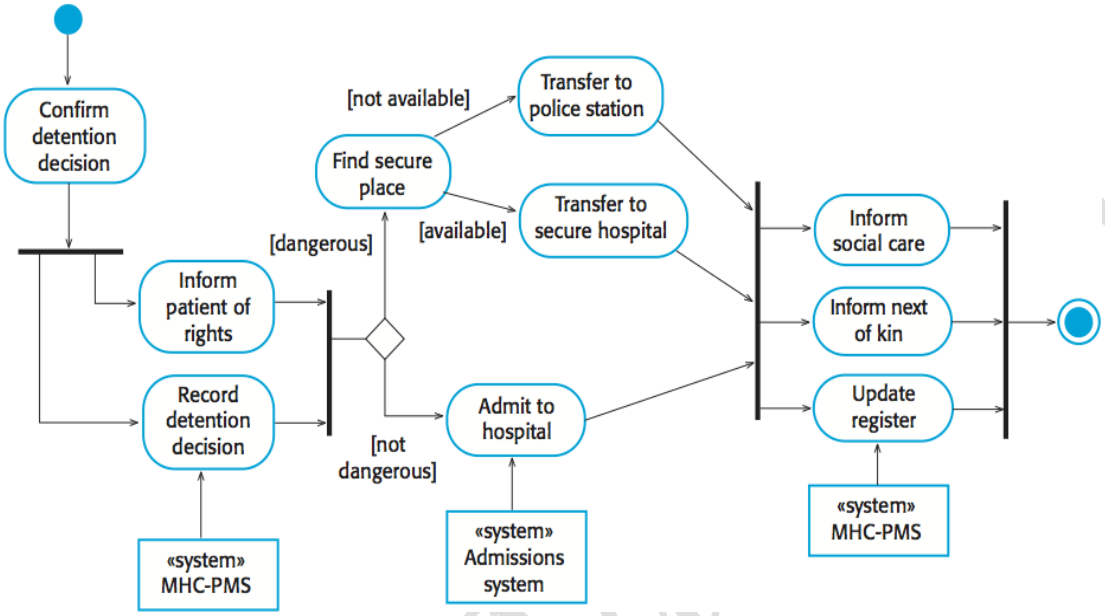
Context and process models

Context models are used to illustrate the operational context of a system - they show what lies outside the system boundaries. Social and organizational concerns may affect the decision on where to position system boundaries. Architectural models show the system and its relationship with other systems.

System boundaries are established to define what is inside and what is outside the system. They show other systems that are used or depend on the system being developed. The position of the system boundary has a profound effect on the system requirements. Defining a system boundary is a political judgment since there may be pressures to develop system boundaries that increase/decrease the influence or workload of different parts of an organization.

Context models simply show the other systems in the environment, not how the system being developed is used in that environment. **Process models** reveal how the system being developed is used in broader business processes. UML activity diagrams may be used to define business process models.

The example below shows a UML **activity diagram** describing the process of involuntary detention and the role of MHC-PMS (mental healthcare patient management system) in it.



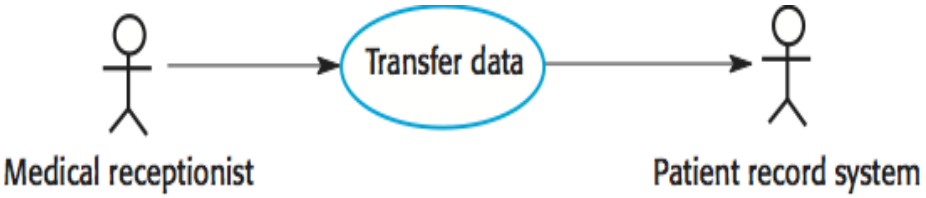
Interaction models

Types of interactions that can be represented in a model:

- Modeling **user interaction** is important as it helps to identify user requirements.
- Modeling **system-to-system interaction** highlights the communication problems that may arise.
- Modeling **component interaction** helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.

Use cases were developed originally to support requirements elicitation and now incorporated into the UML. Each use case represents a discrete task that involves external interaction with a system. Actors in a use case may be people or other systems. Use cases can be represented using a UML use case diagram and in a more detailed textual/tabular format.

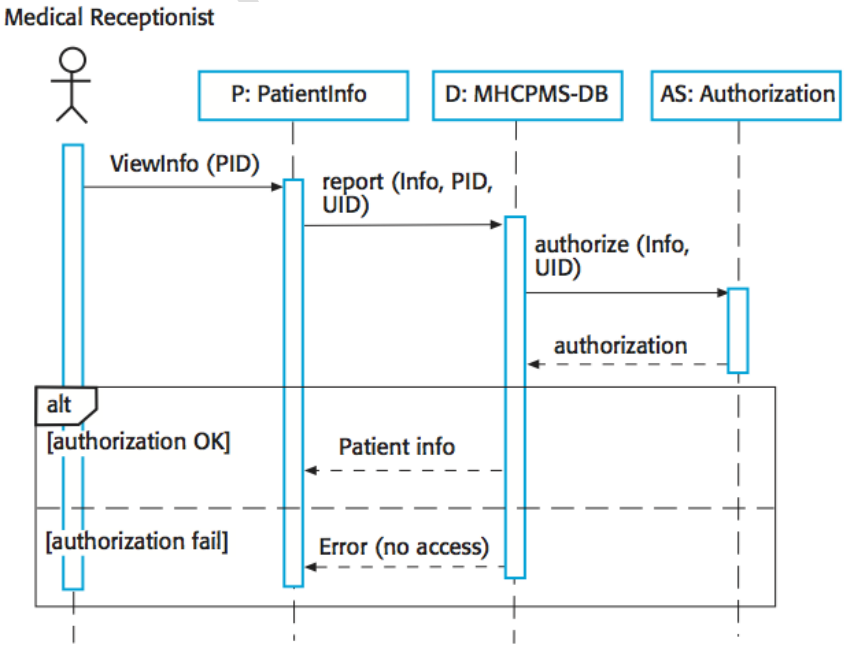
Simple use case diagram:



Use case description in a tabular format:

Use case title	Transfer data
Description	A receptionist may transfer data from the MHC-PMS to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Actor(s)	Medical receptionist, patient records system (PRS)
Preconditions	Patient data has been collected (personal information, treatment summary); The receptionist must have appropriate security permissions to access the patient information and the PRS.
Postconditions	PRS has been updated
Main success scenario	1. Receptionist selects the "Transfer data" option from the menu. 2. PRS verifies the security credentials of the receptionist. 3. Data is transferred. 4. PRS has been updated.
Extensions	2a. The receptionist does not have the necessary security credentials. 2a.1. An error message is displayed. 2a.2. The receptionist backs out of the use case.

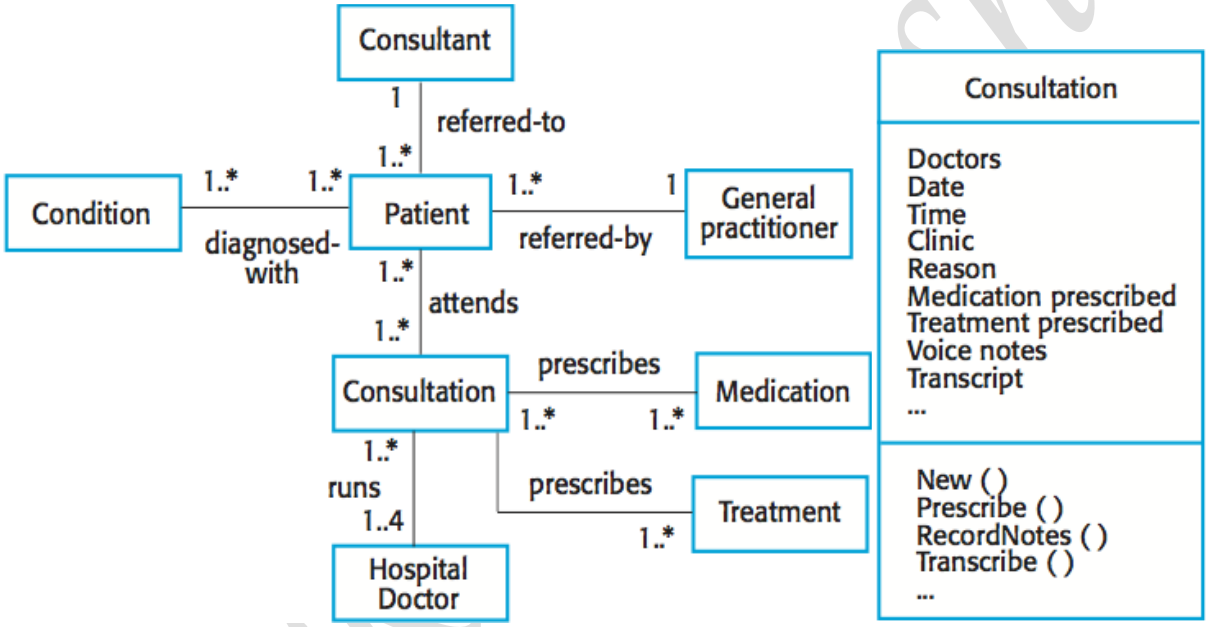
UML **sequence diagrams** are used to model the interactions between the actors and the objects within a system. A sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance. The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these. Interactions between objects are indicated by annotated arrows.



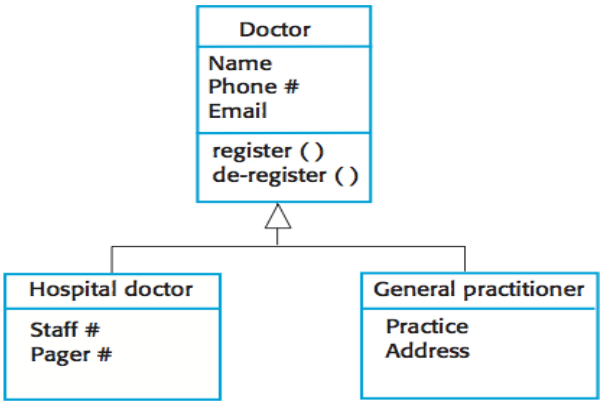
Structural models

Structural models of software display the organization of a system in terms of the components that make up that system and their relationships. Structural models may be **static** models, which show the structure of the system design, or **dynamic** models, which show the organization of the system when it is executing. You create structural models of a system when you are discussing and designing the system architecture.

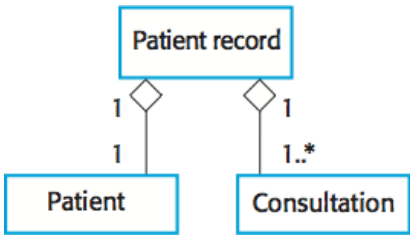
UML **class diagrams** are used when developing an object-oriented system model to show the classes in a system and the associations between these classes. An object class can be thought of as a general definition of one kind of system object. An association is a link between classes that indicates that there is some relationship between these classes. When you are developing models during the early stages of the software engineering process, objects represent something in the real world, such as a patient, a prescription, doctor, etc.



Generalization is an everyday technique that we use to manage complexity. In modeling systems, it is often useful to examine the classes in a system to see if there is scope for generalization. In object-oriented languages, such as Java, generalization is implemented using the class **inheritance** mechanisms built into the language. In a generalization, the attributes and operations associated with higher-level classes are also associated with the lower-level classes. The lower-level classes are subclasses inherit the attributes and operations from their superclasses. These lower-level classes then add more specific attributes and operations.



An **aggregation** model shows how classes that are collections are composed of other classes. Aggregation models are similar to the part-of relationship in semantic data models.

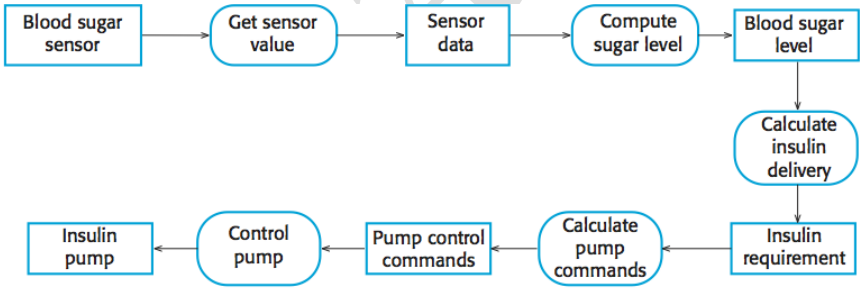


Behavioral models

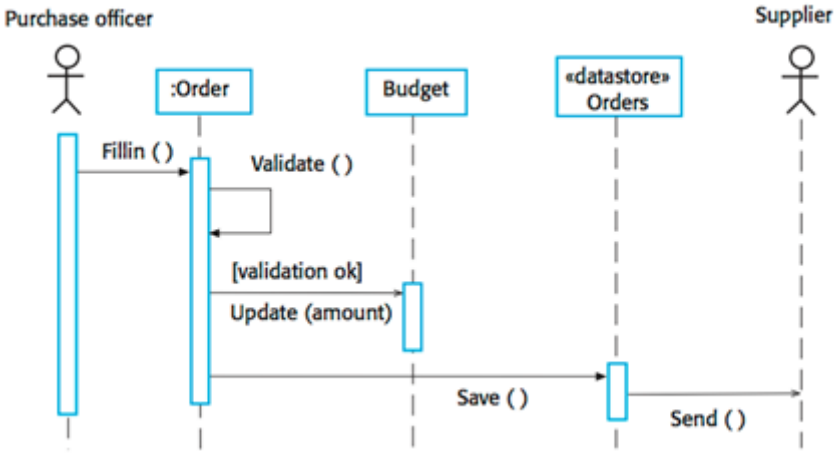
Behavioral models are models of the dynamic behavior of a system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment. Two types of stimuli:

- Some **data** arrives that has to be processed by the system.
- Some **event** happens that triggers system processing. Events may have associated data, although this is not always the case.

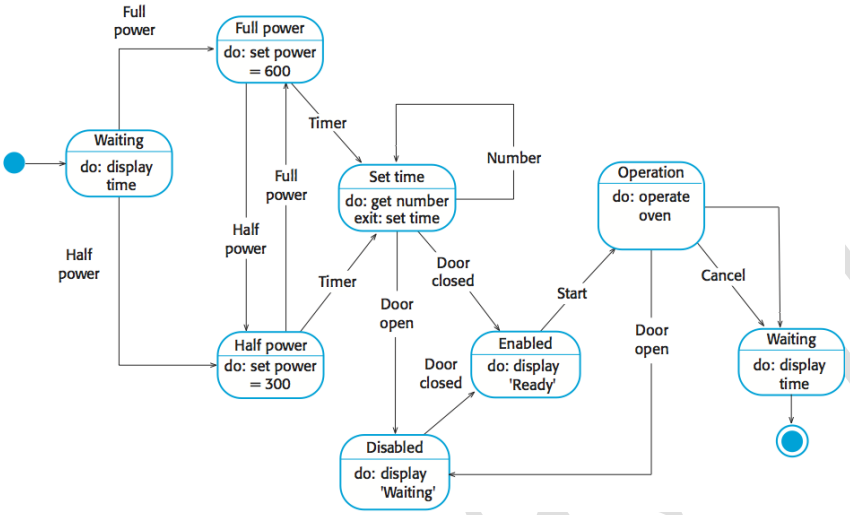
Many business systems are data-processing systems that are primarily driven by data. They are controlled by the data input to the system, with relatively little external event processing. **Data-driven models** show the sequence of actions involved in processing input data and generating an associated output. They are particularly useful during the analysis of requirements as they can be used to show end-to-end processing in a system. Data-driven models can be created using UML **activity diagrams**:



Data-driven models can also be created using UML **sequence diagrams**:



Real-time systems are often event-driven, with minimal data processing. For example, a landline phone switching system responds to events such as 'receiver off hook' by generating a dial tone. **Event-driven models** shows how a system responds to external and internal events. It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another. Event-driven models can be created using UML **state diagrams**:



Architectural Design

Architectural design is a process for identifying the sub-systems making up a system and the framework for sub-system control and communication. The output of this design process is a description of the software architecture. Architectural design is an early stage of the system design process. It represents the link between specification and design processes and is often carried out in parallel with some specification activities. It involves identifying major system components and their communications.

Software architectures can be designed at **two levels of abstraction**:

- **Architecture in the small** is concerned with the architecture of individual programs. At this level, we are concerned with the way that an individual program is decomposed into components.
- **Architecture in the large** is concerned with the architecture of complex enterprise systems that include other systems, programs, and program components. These enterprise systems are distributed over different computers, which may be owned and managed by different companies.

Three **advantages** of explicitly designing and documenting software architecture:

- **Stakeholder communication**: Architecture may be used as a focus of discussion by system stakeholders.
- **System analysis**: Well-documented architecture enables the analysis of whether the system can meet its non-functional requirements.
- **Large-scale reuse**: The architecture may be reusable across a range of systems or entire lines of products.

Software architecture is most often represented using simple, informal **block diagrams** showing entities and relationships. **Pros**: simple, useful for communication with stakeholders, great for project planning. **Cons**: lack of semantics, types of relationships between entities, visible properties of entities in the architecture.

Uses of architectural models:

As a way of facilitating discussion about the system design: A high-level architectural view of a system is useful for communication with system stakeholders and project planning because it is not cluttered with detail. Stakeholders can relate to it and understand an abstract view of the system. They can then discuss the system as a whole without being confused by detail.

As a way of documenting an architecture that has been designed: The aim here is to produce a complete system model that shows the different components in a system, their interfaces and their connections.

Architectural design decisions

Architectural design is a **creative process** so the process differs depending on the type of system being developed. However, a number of **common decisions** span all design processes and these decisions affect the non-functional characteristics of the system:

- Is there a generic application architecture that can be used?
- How will the system be distributed?
- What architectural styles are appropriate?
- What approach will be used to structure the system?
- How will the system be decomposed into modules?
- What control strategy should be used?
- How will the architectural design be evaluated?
- How should the architecture be documented?

Systems in the same domain often have **similar architectures** that reflect domain concepts. Application product lines are built around a core architecture with variants that satisfy particular customer requirements. The architecture of a system may be designed around one of more **architectural patterns/styles**, which capture the essence of an architecture and can be instantiated in different ways.

The particular architectural style should depend on the **non-functional system requirements**:

- **Performance**: localize critical operations and minimize communications. Use large rather than fine-grain components.
- **Security**: use a layered architecture with critical assets in the inner layers.
- **Safety**: localize safety-critical features in a small number of sub-systems.
- **Availability**: include redundant components and mechanisms for fault tolerance.
- **Maintainability**: use fine-grain, replaceable components.

Architectural views

Each architectural model only shows one view or perspective of the system. It might show how a system is decomposed into modules, how the run-time processes interact or the different ways in which system components are distributed across a network. For both design and documentation, you usually need to present multiple views of the software architecture.

4+1 view model of software architecture:

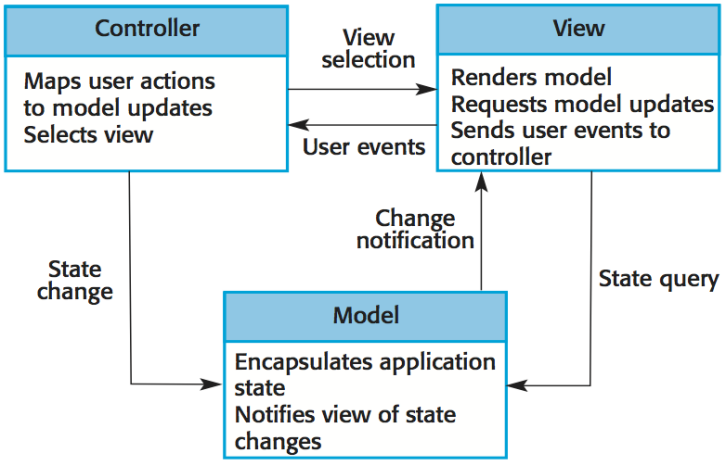
- A **logical** view, which shows the key abstractions in the system as objects or object classes.
- A **process** view, which shows how, at run-time, the system is composed of interacting processes.
- A **development** view, which shows how the software is decomposed for development.
- A **physical** view, which shows the system hardware and how software components are distributed across the processors in the system.
- Related using **use cases** or scenarios (+1).

Architectural patterns

Patterns are a means of representing, sharing and reusing knowledge. An architectural pattern is a **stylized description of a good design practice**, which has been tried and tested in different environments. Patterns should include information about when they are and when the are not useful. Patterns may be represented using tabular and graphical descriptions.

Model-View-Controller

- Serves as a basis of interaction management in many web-based systems.
- Decouples three major interconnected components:
  - The model is the central component of the pattern that directly manages the data, logic and rules of the application. It is the application's dynamic data structure, independent of the user interface.
  - A view can be any output representation of information, such as a chart or a diagram. Multiple views of the same information are possible.
  - The controller accepts input and converts it to commands for the model or view.
- Supported by most language frameworks.

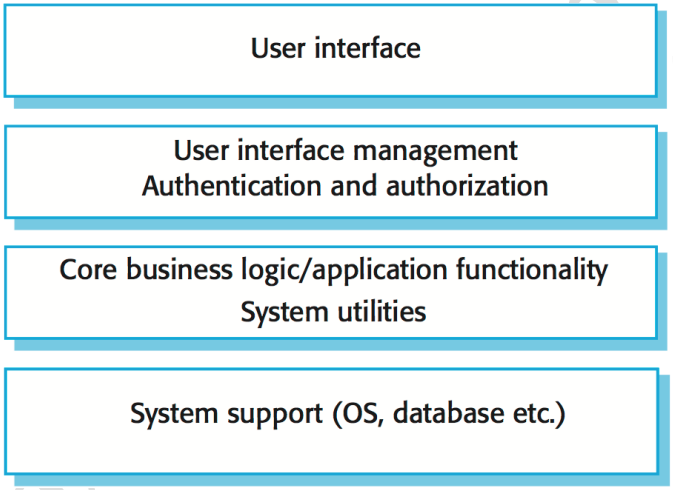


Pattern name	Model-View-Controller (MVC)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model.
Problem description	The display presented to the user frequently changes over time in response to input or computation. Different users have different needs for how they want to view the program.s information. The system needs to reflect data changes to all users in the way that they want to view them, while making it easy to make changes to the user interface.
Solution description	This involves separating the data being manipulated from the manipulation logic and the details of display using three components: Model (a problem-domain component with data and operations, independent of the user interface), View (a data display component), and Controller (a component that receives and acts on user input).
Consequences	Advantages: views and controllers can be easily be added, removed, or changed; views can be added or changed during execution; user interface components can be

	changed, even at runtime. Disadvantages: views and controller are often hard to separate; frequent updates may slow data display and degrade user interface performance; the MVC style makes user interface components (views, controllers) highly dependent on model components.

Layered architecture

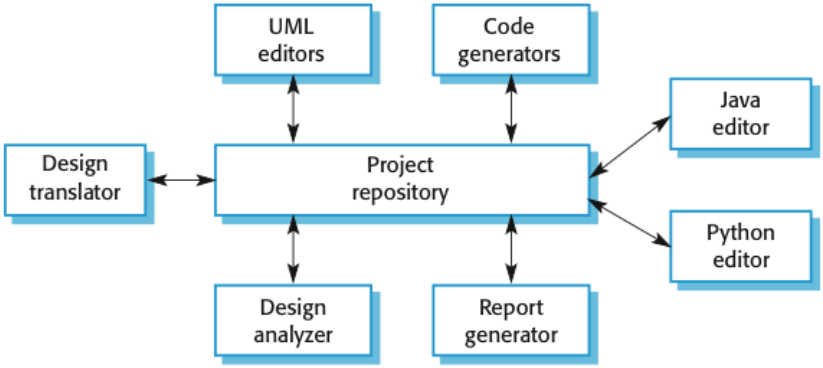
- Used to model the interfacing of sub-systems.
- Organizes the system into a set of layers (or abstract machines) each of which provide a set of services.
- Supports the incremental development of sub-systems in different layers. When a layer interface changes, only the adjacent layer is affected.
- However, often artificial to structure systems in this way.



Name	Layered architecture
Description	Organizes the system into layers with related functionality associated with each layer. A layer provides services to the layer above it so the lowest-level layers represent core services that are likely to be used throughout the system.
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multi-level security.
Advantages	Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

Repository architecture

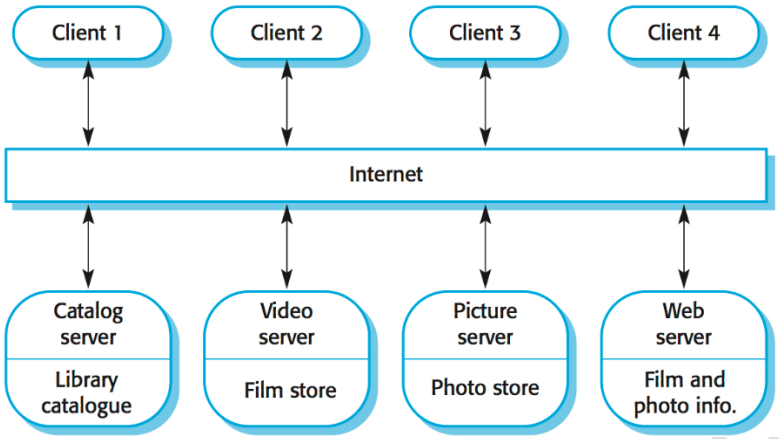
- Sub-systems must exchange data. This may be done in two ways:
  - Shared data is held in a central database or repository and may be accessed by all sub-systems;
  - Each sub-system maintains its own database and passes data explicitly to other sub-systems.
- When large amounts of data are to be shared, the repository model of sharing is most commonly used a this is an efficient data sharing mechanism.



Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent--they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

Client-server architecture

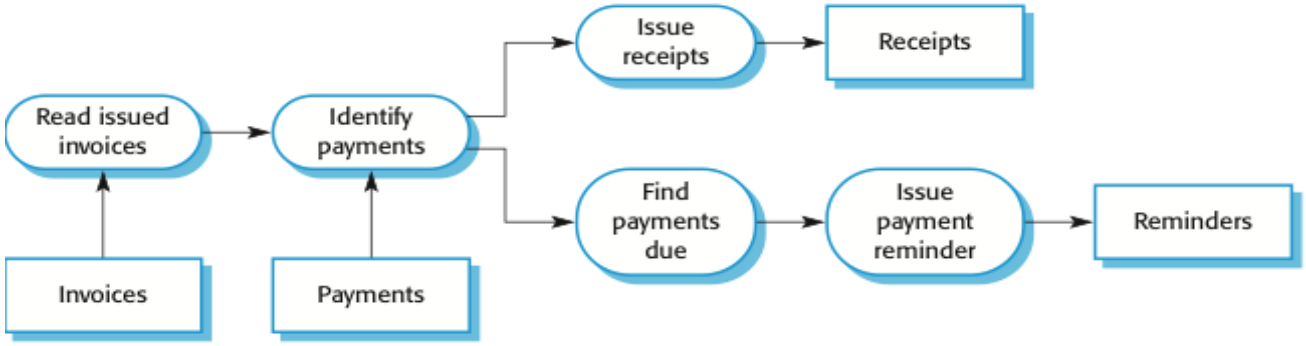
- Distributed system model which shows how data and processing is distributed across a range of components, but can also be implemented on a single computer.
- Set of stand-alone servers which provide specific services such as printing, data management, etc.
- Set of clients which call on these services.
- Network which allows clients to access servers.



Name	Client-server
Description	In a client-server architecture, the functionality of the system is organized into services, with each service delivered from a separate server. Clients are users of these services and access servers to make use of them.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure so susceptible to denial-of-service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. May be management problems if servers are owned by different organizations.

Pipe and filter architecture

- Functional transformations process their inputs to produce outputs.
- May be referred to as a pipe and filter model (as in UNIX shell).
- Variants of this approach are very common. When transformations are sequential, this is a batch sequential model which is extensively used in data processing systems.
- Not really suitable for interactive systems.



Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
When used	Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed upon between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse functional transformations that use incompatible data structures.

Application architectures

Application systems are designed to meet an organizational need. As businesses have much in common, their application systems also tend to have a common architecture that reflects the application requirements. A **generic application architecture** is an architecture for a type of software system that may be configured and adapted to create a system that meets specific requirements. application architectures can be used as a:

- Starting point for architectural design.
- Design checklist.
- Way of organizing the work of the development team.
- Means of assessing components for reuse.
- Vocabulary for talking about application types.

Examples of **application types**:

Data processing applications: Data driven applications that process data in batches without explicit user intervention during the processing.

Transaction processing applications: Data-centred applications that process user requests and update information in a system database.

Event processing systems: Applications where system actions depend on interpreting events from the system's environment.

Language processing systems: Applications where the users' intentions are specified in a formal language that is processed and interpreted by the system.

Design and Implementation

Software design and implementation is the stage in the software engineering process at which an executable software system is developed. **Software design** is a creative activity in which you identify software components and their relationships, based on a customer's requirements. **Implementation** is the process of realizing the design as a program. These two activities are invariably inter-leaved.

In a wide range of domains, it is now possible to buy **commercial off-the-shelf systems** (COTS) that can be adapted and tailored to the users' requirements. When you develop an application in this way, the design process becomes concerned with how to use the configuration features of that system to deliver the system requirements.

Object-oriented design using the UML

Structured object-oriented design processes involve developing a number of different **system models**. They require a lot of effort for development and maintenance and, for small systems, this may not be cost-effective. However, for **large systems** developed by different groups design models are an important communication mechanism. Common activities in these processes include:

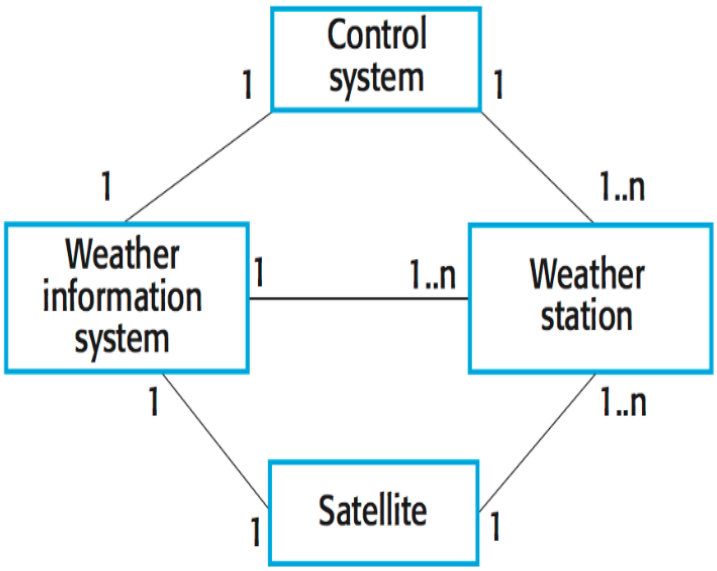
- Define the context and modes of use of the system;
- Design the system architecture;
- Identify the principal system objects;
- Develop design models;
- Specify object interfaces.

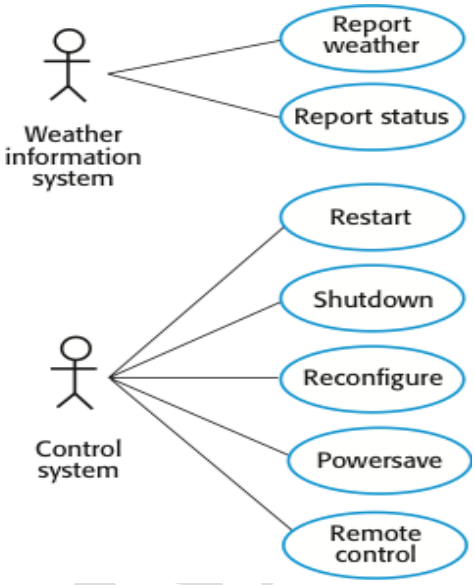
System context and interactions

Understanding the relationships between the software that is being designed and its **external environment** is essential for deciding how to provide the required system functionality and how to structure the system to communicate with its environment. Understanding of the context also lets you establish the **boundaries** of the system. Setting the system boundaries helps you decide what features are implemented in the system being designed and what features are in other associated systems.

A system **context** is a **structural model** (e.g., a class diagram) that demonstrates the other systems in the environment of the system being developed.

An **interaction** model is a **dynamic model** (e.g., a use case diagram + structured natural language description) that shows how the system interacts with its environment as it is used.

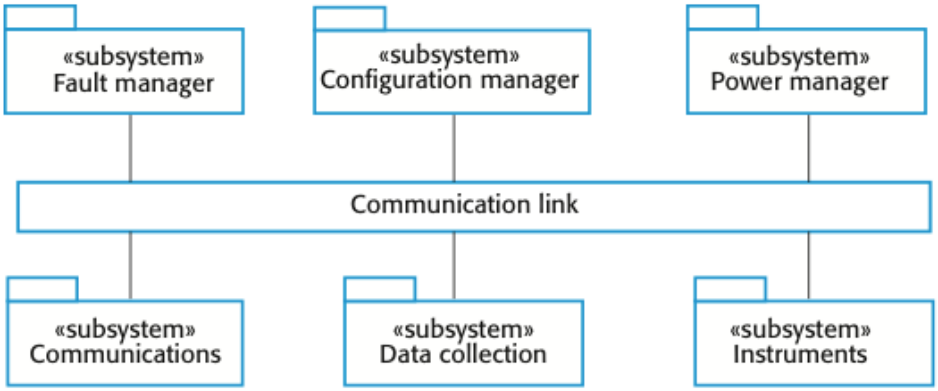




System	Weather station
Use case	Report weather
Actors	Weather information system, Weather station
Description	The weather station sends a summary of the weather data that has been collected from the instruments in the collection period to the weather information system. The data sent are the maximum, minimum, and average ground and air temperatures; the maximum, minimum, and average air pressures; the maximum, minimum, and average wind speeds; the total rainfall; and the wind direction as sampled at five-minute intervals.
Stimulus	The weather information system establishes a satellite communication link with the weather station and requests transmission of the data.
Response	The summarized data is sent to the weather information system.
Comments	Weather stations are usually asked to report once per hour but this frequency may differ from one station to another and may be modified in the future.

Architectural design

Once interactions between the system and its environment have been understood, you use this information for designing the system architecture. You identify the **major components** that make up the system and **their interactions**, and then may organize the components using an architectural pattern (e.g. a layered or client-server model).



Identifying object classes is often a difficult part of object oriented design. There is no 'magic formula' for object identification. It relies on the skill, experience and domain knowledge of system designers. Object identification is an iterative process.

You are unlikely to get it right first time. **Approaches** to object identification include:

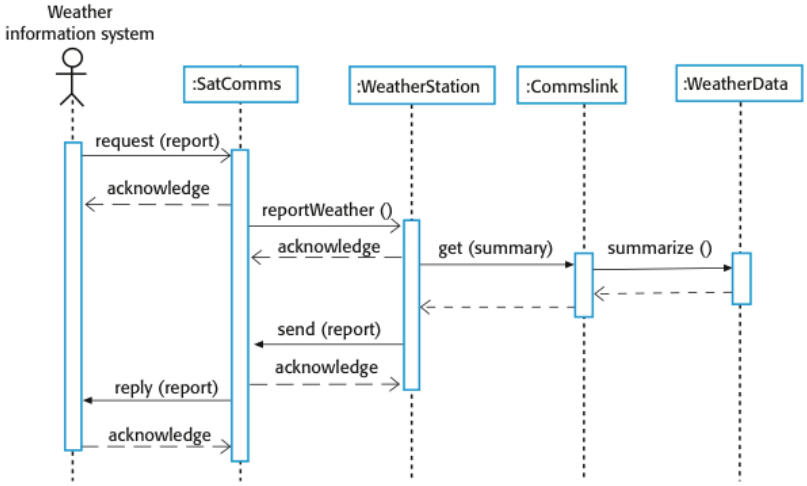
- Use a **grammatical approach** based on a natural language description of the system.
- Base the identification on **tangible things** in the application domain.
- Use a **behavioral approach** and identify objects based on what participates in what behavior.
- Use a **scenario-based analysis**. The objects, attributes and methods in each scenario are identified.

Design models

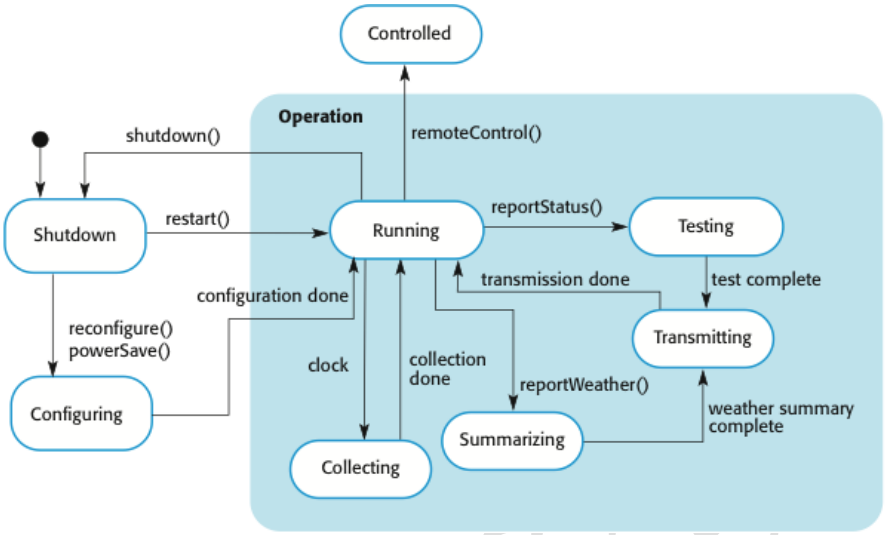
Design models show the objects and object classes and relationships between these entities. **Static models** describe the static structure of the system in terms of object classes and relationships. **Dynamic models** describe the dynamic interactions between objects.

Subsystem models show logical groupings of objects into coherent subsystems. These are represented using a form of class diagram with each subsystem shown as a package with enclosed objects. Subsystem models are static (structural) models.

Sequence models show the sequence of object interactions. These are represented using a UML sequence or a collaboration diagram. Sequence models are dynamic models.



State machine models show how individual objects change their state in response to events. These are represented in the UML using state diagrams. State machine models are dynamic models. State diagrams are useful high-level models of a system or an object's run-time behavior.



Interface specification

Object interfaces have to be specified so that the objects and other components can be **designed in parallel**. Designers should avoid designing the interface representation but should hide this in the object itself. Objects may have several interfaces which are viewpoints on the methods provided. The UML uses class diagrams for interface specification but Java may also be used.

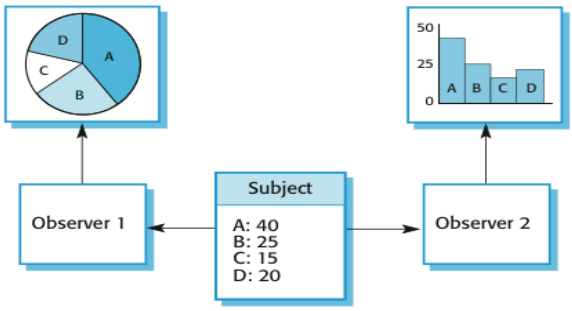
Design patterns

A design pattern is a way of **reusing abstract knowledge** about a problem and its solution. A pattern is a description of the problem and the essence of its solution. It should be sufficiently abstract to be reused in different settings. Pattern descriptions usually make use of object-oriented characteristics such as inheritance and polymorphism.

Design pattern elements:

- **Name:** A meaningful pattern identifier
- **Problem description:** A common situation where this pattern is applicable
- **Solution description:** Not a concrete design but a template for a design solution that can be instantiated in different ways
- **Consequences:** The results and trade-offs of applying the pattern

Example: the Observer pattern



Pattern name	Observer
Description	Separates the display of the state of an object from the object itself and allows alternative displays to be provided. When the object state changes, all displays are automatically notified and updated to reflect the change.
Problem description	In many situations, you have to provide multiple displays of state information, such as a graphical display and a tabular display. Not all of these may be known when the information is specified. All alternative presentations should support interaction and, when the state is changed, all displays must be updated. This pattern may be used in all situations where more than one display format for state information is required and where it is not necessary for the object that maintains the state information to know about the specific display formats used.
Solution description	This involves two abstract objects, Subject and Observer, and two concrete objects, ConcreteSubject and ConcreteObject, which inherit the attributes of the related abstract objects. The abstract objects include general operations that are applicable in all situations. The state to be displayed is maintained in ConcreteSubject, which inherits operations from Subject allowing it to add and remove Observers (each observer corresponds to a display) and to issue a notification when the state has changed. The ConcreteObserver maintains a copy of the state of ConcreteSubject and implements the Update() interface of Observer that allows these copies to be kept in step. The ConcreteObserver automatically displays the state and reflects changes whenever the state is updated.
Consequences	The subject only knows the abstract Observer and does not know details of the concrete class. Therefore there is minimal coupling between these objects. Because of this lack of knowledge, optimizations that enhance display performance are impractical. Changes to the subject may cause a set of linked updates to observers to be generated, some of which may not be necessary.

Reuse

From the 1960s to the 1990s, most new software was developed from scratch, by writing all code in a high-level programming language. The only significant reuse or software was the reuse of functions and objects in programming language libraries. Costs and schedule pressure mean that this approach became increasingly unviable, especially for commercial and Internet-based systems. An approach to development based around the reuse of existing software emerged and is now generally used for business and scientific software.

Levels of reuse:

- The **abstraction** level: don't reuse software directly but use knowledge of successful abstractions in the software design.
- The **object** level: directly reuse objects from a library rather than writing the code yourself.

- The **component** level: components (collections of objects and object classes) are reused in application systems.
- The **system** level: entire application systems are reused.

Costs of reuse:

- The costs of the **time** spent in looking for software to reuse and assessing whether or not it meets your needs.
- Where applicable, the costs of **buying** the reusable software. For large off-the-shelf systems, these costs can be very high.
- The costs of **adapting and configuring** the reusable software components or systems to reflect the requirements of the system that you are developing.
- The costs of **integrating** reusable software elements with each other (if you are using software from different sources) and with the new code that you have developed.

Configuration management

Configuration management is the name given to the general process of **managing a changing software system**. The aim of configuration management is to support the system integration process so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system. Configuration management activities include:

- **Version management**, where support is provided to keep track of the different versions of software components. Version management systems include facilities to coordinate development by several programmers.
- **System integration**, where support is provided to help developers define what versions of components are used to create each version of a system. This description is then used to build a system automatically by compiling and linking the required components.
- **Problem tracking**, where support is provided to allow users to report bugs and other problems, and to allow all developers to see who is working on these problems and when they are fixed.

Host-target development

Most software is developed on one computer (the host, **development platform**), but runs on a separate machine (the target, **execution platform**). A platform is more than just hardware; it includes the installed operating system plus other supporting software such as a database management system or, for development platforms, an interactive development environment (IDE). Development platform usually has different installed software than execution platform; these platforms may have different architectures. Mobile app development (e.g. for Android) is a good example.

Typical development platform tools include:

- An integrated compiler and syntax-directed editing system that allows you to create, edit and compile code.
- A language debugging system.
- Graphical editing tools, such as tools to edit UML models.

- Testing tools, such as JUnit that can automatically run a set of tests on a new version of a program.
- Project support tools that help you organize the code for different development projects.

Open-source development

Open-source development is an approach to software development in which the **source code** of a software system is **published** and **volunteers** are invited to **participate in the development process**. Its roots are in the Free Software Foundation, which advocates that source code should not be proprietary but rather should always be available for users to examine and modify as they wish. Open source software extended this idea by using the Internet to recruit a much larger population of volunteer developers. Many of them are also users of the code.

The best-known open source product is, of course, the Linux operating system which is widely used as a server system and, increasingly, as a desktop environment. Other important open source products are Java, the Apache web server and the mySQL database management system.

A **fundamental principle** of open-source development is that **source code** should be **freely available**, this does not mean that anyone can do as they wish with that code. Typical **licensing models** include:

- The GNU **General Public License** (GPL). This is a so-called 'reciprocal' license that means that if you use open source software that is licensed under the GPL license, then you must make that software open source.
- The GNU **Lesser General Public License** (LGPL) is a variant of the GPL license where you can write components that link to open source code without having to publish the source of these components.
- The **Berkley Standard Distribution** (BSD) License. This is a non-reciprocal license, which means you are not obliged to re-publish any changes or modifications made to open source code. You can include the code in proprietary systems that are sold.

Software Testing

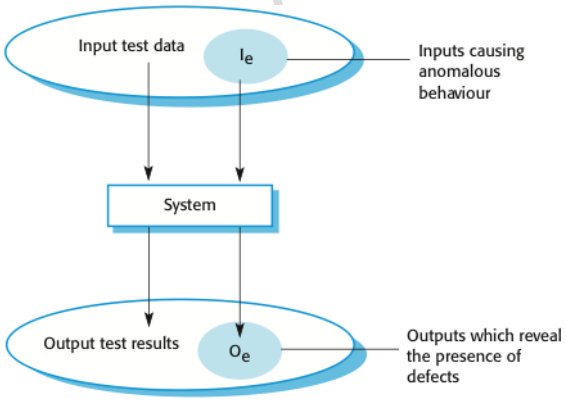
Testing is intended to show that a **program does what it is intended to do** and to **discover program defects** before it is put into use. When you test software, you execute a program using artificial data. You check the results of the test run for errors, anomalies or information about the program's non-functional attributes. **Testing can reveal the presence of errors, but NOT their absence.** Testing is part of a more general verification and validation process, which also includes static validation techniques.

Goals of software testing:

- To **demonstrate** to the developer and the customer that the **software meets its requirements**.
  - Leads to **validation testing**: you expect the system to perform correctly using a given set of test cases that reflect the system's expected use.
  - A successful test shows that the system operates as intended.

- To **discover** situations in which the behavior of the software is **incorrect, undesirable or does not conform to its specification**.
  - Leads to **defect testing**: the test cases are designed to expose defects; the test cases can be deliberately obscure and need not reflect how the system is normally used.
  - A successful test is a test that makes the system perform incorrectly and so exposes a defect in the system.

Testing can be viewed as an **input-output process**:



Verification and validation

Testing is part of a broader process of software **verification and validation** (V & V).

- **Verification: Are we building the product right?**
  - The software should conform to its specification.
- **Validation: Are we building the right product?**
  - The software should do what the user really requires.

The **goal of V & V** is to establish confidence that the system is **good enough for its intended use**, which depends on:

- **Software purpose**: the level of confidence depends on how critical the software is to an organization.
- **User expectations**: users may have low expectations of certain kinds of software.
- **Marketing environment**: getting a product to market early may be more important than finding defects in the program.

Inspections and testing

Software inspections involve people examining the source representation with the aim of discovering anomalies and defects. Inspections not require execution of a system so may be used before implementation. They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.). They have been shown to be an effective technique for discovering program errors.

Advantages of inspections include:

- During testing, errors can mask (hide) other errors. Because inspection is a static process, you don't have to be concerned with **interactions between errors**.
- **Incomplete versions** of a system can be inspected without additional costs. If a program is incomplete, then you need to develop specialized test harnesses to test the parts that are available.
- As well as searching for program defects, an inspection can also consider **broader quality attributes** of a program, such as compliance with standards, portability and maintainability.

Inspections and testing are complementary and not opposing verification techniques. Both should be used during the V & V process. Inspections can check conformance with a specification but not conformance with the customer's real requirements. Inspections cannot check non-functional characteristics such as performance, usability, etc.

Typically, a commercial software system has to go through **three stages of testing**:

- **Development testing**: the system is tested during development to discover bugs and defects.
- **Release testing**: a separate testing team test a complete version of the system before it is released to users.
- **User testing**: users or potential users of a system test the system in their own environment.

Development testing

Development testing includes all testing activities that are carried out by the team developing the system:

- **Unit testing**: individual program units or object classes are tested; should focus on testing the functionality of objects or methods.
- **Component testing**: several individual units are integrated to create composite components; should focus on testing component interfaces.
- **System testing**: some or all of the components in a system are integrated and the system is tested as a whole; should focus on testing component interactions.

Unit testing

Unit testing is the process of **testing individual components in isolation**. It is a defect testing process. Units may be:

- **Individual functions** or methods within an object;
- **Object classes** with several attributes and methods;
- **Composite components** with defined interfaces used to access their functionality.

When **testing object classes**, tests should be designed to provide coverage of all of the features of the object:

- Test **all operations** associated with the object;
- Set and check the **value of all attributes** associated with the object;
- Put the object into **all possible states**, i.e. simulate all events that cause a state change.

Whenever possible, **unit testing should be automated** so that tests are run and checked without manual intervention. In automated unit testing, you make use of a test automation framework (such as JUnit) to write and run your program tests. Unit testing frameworks provide generic test classes that you extend to create specific test cases. They can then run all of the tests that you have implemented and report, often through some GUI, on the success of otherwise of the tests. An automated test has three parts:

- A **setup** part, where you initialize the system with the test case, namely the inputs and expected outputs.
- A **call** part, where you call the object or method to be tested.
- An **assertion** part where you compare the result of the call with the expected result. If the assertion evaluates to true, the test has been successful if false, then it has failed.

The test cases should show that, when used as expected, the component that you are testing does what it is supposed to do. If there are defects in the component, these should be revealed by test cases. This leads to **two types of unit test cases**:

- The first of these should reflect **normal operation** of a program and should show that the component works as expected.
- The other kind of test case should be based on testing experience of where common problems arise. It should use **abnormal inputs** to check that these are properly processed and do not crash the component.

Component testing

Software components are often composite components that are **made up of several interacting objects**. You access the functionality of these objects through the **defined component interface**. Testing composite components should therefore focus on showing that the component interface behaves according to its specification. Objectives are to detect faults due to interface errors or invalid assumptions about interfaces. Interface types include:

- **Parameter interfaces**: data passed from one method or procedure to another.
- **Shared memory interfaces**: block of memory is shared between procedures or functions.

- **Procedural interfaces**: sub-system encapsulates a set of procedures to be called by other sub-systems.
- **Message passing interfaces**: sub-systems request services from other sub-systems.

Interface errors:

- **Interface misuse**: a calling component calls another component and makes an error in its use of its interface e.g. parameters in the wrong order.
- **Interface misunderstanding**: a calling component embeds assumptions about the behavior of the called component which are incorrect.
- **Timing errors**: the called and the calling component operate at different speeds and out-of-date information is accessed.

General guidelines for interface testing:

- Design tests so that parameters to a called procedure are at the extreme ends of their ranges.
- Always test pointer parameters with null pointers.
- Design tests which cause the component to fail.
- Use stress testing in message passing systems.
- In shared memory systems, vary the order in which components are activated.

System testing

System testing during development involves **integrating components** to create a version of the system and then **testing the integrated system**. The focus in system testing is **testing the interactions between components**. System testing checks that components are compatible, interact correctly and transfer the right data at the right time across their interfaces. System testing tests the **emergent behavior** of a system.

During system testing, reusable components that have been separately developed and off-the-shelf systems may be integrated with newly developed components. The complete system is then tested. Components developed by different team members or sub-teams may be integrated at this stage. System testing is a collective rather than an individual process.

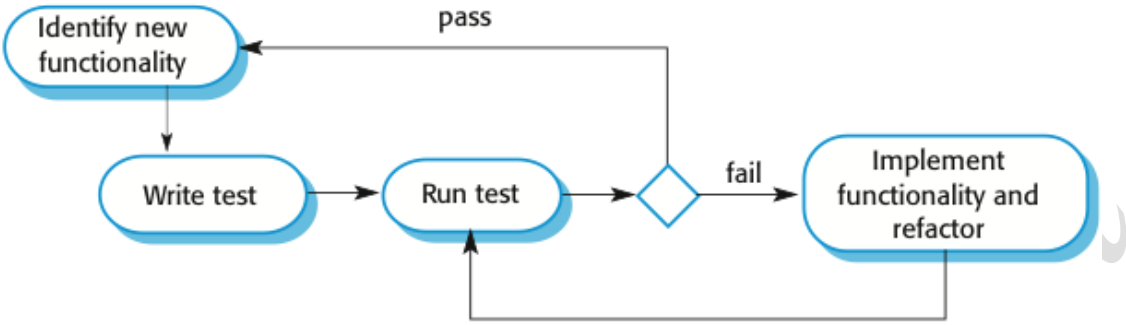
The **use cases** developed to identify system interactions can be used as a **basis for system testing**. Each use case usually involves several system components so testing the use case forces these interactions to occur. The **sequence diagrams** associated with the use case **document the components and their interactions** that are being tested.

Test-driven development

Test-driven development (TDD) is an approach to program development in which you **inter-leave testing and code development**. **Tests are written before code** and 'passing' the tests is the critical driver of development. This is a differentiating feature of TDD versus writing unit tests after the code is written: it makes the developer focus on the requirements before writing the code. **The code is developed incrementally, along with a test for that increment**. You don't move on to the next increment until the code that you have developed passes its test. TDD was introduced as part of agile methods such as Extreme Programming. However, it can also be used in plan-driven development processes.

TDD example - a string calculator.

The **goal of TDD** isn't to ensure we write tests by writing them first, but to **produce working software that achieves a targeted set of requirements using simple, maintainable solutions**. To achieve this goal, TDD provides strategies for **keeping code working, simple, relevant, and free of duplication**.



TDD process includes the following activities:

1. Start by **identifying the increment of functionality** that is required. This should normally be small and implementable in a few lines of code.
2. **Write a test** for this functionality and implement this as an automated test.
3. **Run the test**, along with all other tests that have been implemented. Initially, you have not implemented the functionality so the new test will fail.
4. **Implement the functionality and re-run the test**.
5. Once all tests run successfully, you move on to implementing the next chunk of functionality.

Benefits of test-driven development:

- **Code coverage**: every code segment that you write has at least one associated test so all code written has at least one test.
- **Regression testing**: a regression test suite is developed incrementally as a program is developed.
- **Simplified debugging**: when a test fails, it should be obvious where the problem lies; the newly written code needs to be checked and modified.
- **System documentation**: the tests themselves are a form of documentation that describe what the code should be doing.

Regression testing is testing the system to check that **changes have not 'broken' previously working code**. In a manual testing process, regression testing is expensive but, with automated testing, it is simple and straightforward. All tests are rerun every time a change is made to the program. Tests must run 'successfully' before the change is committed.

Release testing

Release testing is the process of **testing a particular release** of a system that is **intended for use outside of the development team**. The primary goal of the release testing process is to **convince the customer of the system that it is good enough for use**. Release testing, therefore, has to show that the system delivers its specified functionality, performance and dependability, and that it does not fail during normal use. Release testing is usually a black-box testing process where **tests are only derived from the system specification**.

Release testing is a form of system testing. Important differences:

- A separate team that has not been involved in the system development, should be responsible for release testing.
- System testing by the development team should focus on discovering bugs in the system (defect testing). The objective of release testing is to check that the system meets its requirements and is good enough for external use (validation testing).

Requirements-based testing involves examining each requirement and developing a test or tests for it. It is validation rather than defect testing: you are trying to demonstrate that the system has properly implemented its requirements.

Scenario testing is an approach to release testing where you devise typical scenarios of use and use these to develop test cases for the system. Scenarios should be realistic and real system users should be able to relate to them. If you have used scenarios as part of the requirements engineering process, then you may be able to reuse these as testing scenarios.

Part of release testing may involve testing the **emergent properties** of a system, such as performance and reliability. Tests should reflect the profile of use of the system. Performance tests usually involve planning a series of tests where the load is steadily increased until the system performance becomes unacceptable. Stress testing is a form of performance testing where the system is deliberately overloaded to test its failure behavior.

User testing

User or customer testing is a stage in the testing process in which **users or customers provide input and advice on system testing**. User testing is essential, even when comprehensive system and release testing have been carried out. Types of user testing include:

- **Alpha testing**: users of the software work with the development team to test the software at the developer's site.
- **Beta testing**: a release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
- **Acceptance testing**: customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment.

In agile methods, the user/customer is part of the development team and is responsible for making decisions on the acceptability of the system. Tests are defined by the user/customer and are integrated with other tests in that they are run automatically when changes are made. Main problem here is whether or not the embedded user is 'typical' and can represent the interests of all system stakeholders.

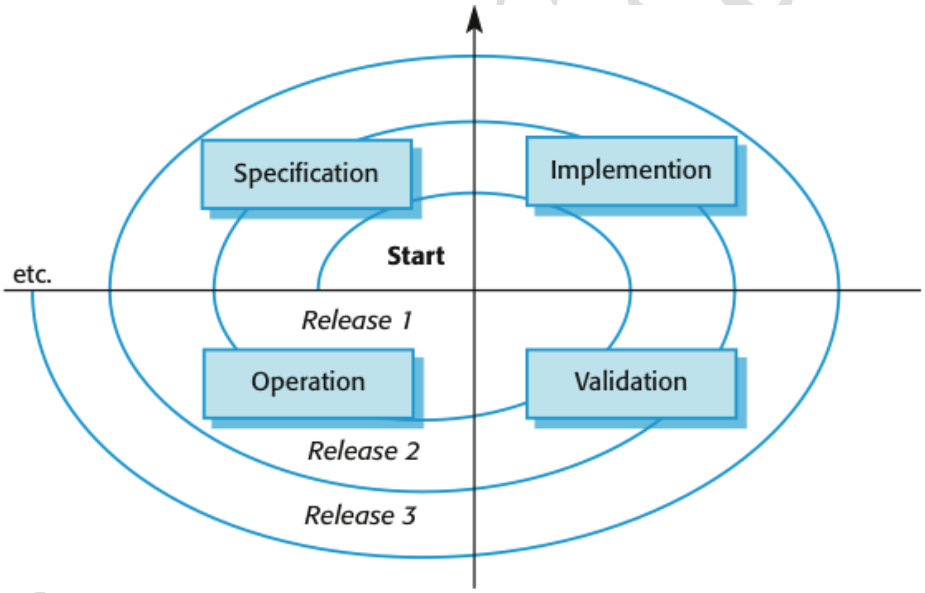
Software Evolution

There are many reasons why **software change is inevitable**:

- New requirements emerge when the software is used;
- The business environment changes;
- Errors must be repaired;
- New computers and equipment is added to the system;
- The performance or reliability of the system may have to be improved.

A key problem for all organizations is implementing and managing change to their existing software systems.

Organizations have **huge investments in their software systems** - they are critical business assets. To maintain the value of these assets to the business, they must be changed and updated. The majority of the software budget in large companies is devoted to changing and evolving existing software rather than developing new software. A **spiral model** of development and evolution represents how a **software system evolves through a sequence of multiple releases**.

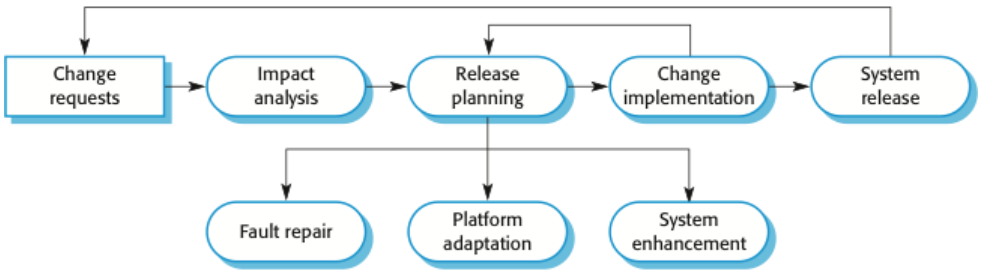


Evolution processes

Software evolution processes depend on:

- The **type of software** being maintained;
- The **development processes** used;
- The **skills and experience** of the people involved.

Proposals for change are the driver for system evolution. These should be linked with components that are affected by the change, thus allowing the cost and impact of the change to be estimated. Change identification and evolution continues throughout the system lifetime.



Change implementation can be viewed as an iteration of the development process where the revisions to the system are designed, implemented and tested. A critical difference is that the first stage of change implementation may involve program understanding, especially if the original system developers are not responsible for the change implementation. During the program understanding phase, you have to understand how the program is structured, how it delivers functionality and how the proposed change might affect the program.

Agile methods are based on incremental development so the transition from development to evolution is a seamless one. **Evolution** is simply a **continuation of the development process** based on frequent system releases. Automated regression testing is particularly valuable when changes are made to a system. Changes may be expressed as additional user stories.

Program evolution dynamics

Program evolution dynamics is the **study of the processes of system change**. The **system requirements** are likely to change while the system is being developed because the environment is changing, therefore a delivered system won't meet its requirements. Systems are tightly coupled with their environment. When a system is installed in an environment it changes that environment and therefore changes the system requirements. Systems MUST be changed if they are to remain useful in an environment.

After several major empirical studies, Lehman and Belady proposed that there were a number of **'laws'** which **apply to all systems as they evolved**. There are sensible observations rather than laws. They are applicable to large systems developed by large organizations.

Law	**Description**
Continuing change	A program that is used in a real-world environment must necessarily change, or else become progressively less useful in that environment.
Increasing complexity	As an evolving program changes, its structure tends to become more complex. Extra resources must be devoted to preserving and simplifying the structure.
Large program evolution	Program evolution is a self-regulating process. System attributes such as size, time between releases, and the number of reported errors is approximately invariant for each system release.
Organizational stability	Over a program's lifetime, its rate of development is approximately constant and independent of the resources devoted to system development.
Conservation of familiarity	Over the lifetime of a system, the incremental change in each release is approximately constant.

Continuing growth	The functionality offered by systems has to continually increase to maintain user satisfaction.
Declining quality	The quality of systems will decline unless they are modified to reflect changes in their operational environment.
Feedback system	Evolution processes incorporate multi agent, multi loop feedback systems and you have to treat them as feedback systems to achieve significant product improvement.

Software maintenance

Software maintenance focuses on **modifying a program after it has been put into use**. The term is mostly used for changing custom software. Generic software products are said to evolve to create new versions. Maintenance does not normally involve major changes to the system's architecture. Changes are implemented by modifying existing components and adding new components to the system.

Types of software maintenance include:

- Maintenance to **repair software faults**: changing a system to correct deficiencies in the way meets its requirements.
- Maintenance to **adapt software to a different operating environment**: changing a system so that it operates in a different environment (computer, OS, etc.) from its initial implementation.
- Maintenance to **add to or modify the system's functionality**: modifying the system to satisfy new requirements.

Maintenance costs are usually greater than development costs (2x to 100x depending on the application). Costs are affected by both technical and non-technical factors; they tend to increase as software is maintained. Maintenance corrupts the software structure making further maintenance more difficult. Aging software can have high support costs (e.g. old languages, compilers etc.).

Maintenance cost factors include:

- **Team stability**: maintenance costs are reduced if the same staff are involved with them for some time.
- **Contractual responsibility**: the developers of a system may have no contractual responsibility for maintenance so there is no incentive to design for future change.
- **Staff skills**: maintenance staff are often inexperienced and have limited domain knowledge.
- **Program age and structure**: as programs age, their structure is degraded and they become harder to understand and change.

Maintenance prediction is concerned with assessing which parts of the system may cause problems and have high maintenance costs. Predicting the number of changes requires and understanding of the relationships between a system and its environment. Tightly coupled systems require changes whenever the environment is changed. Factors influencing this relationship are:

- Number and complexity of **system interfaces**;

- Number of inherently **volatile system requirements**;
- The **business processes** where the system is used.

Predictions of maintainability can be made by **assessing the complexity** of system components. Studies have shown that most maintenance effort is spent on a relatively small number of system components. Complexity depends on:

- Complexity of control structures;
- Complexity of data structures;
- Object, method (procedure) and module size.

Process metrics may be used to assess maintainability; if any or all of these is increasing, this may indicate a decline in maintainability:

- Number of requests for corrective maintenance;
- Average time required for impact analysis;
- Average time taken to implement a change request;
- Number of outstanding change requests.

System reengineering refers to restructuring or rewriting part or all of a legacy system **without changing its functionality**. It is applicable where some but not all sub-systems of a larger system require frequent maintenance. Reengineering involves adding effort to make them easier to maintain. The system may be restructured and re-documented. **Advantages** of reengineering include:

- **Reduced risk**: there is a high risk in new software development. There may be development problems, staffing problems and specification problems.
- **Reduced cost**: the cost of reengineering is often significantly less than the costs of developing new software.

Reengineering process **activities** include:

- **Source code translation**: convert code to a new language;
- **Reverse engineering**: analyze the program to understand it;
- **Program structure improvement**: restructure automatically for understandability;
- **Program modularization**: reorganize the program structure;
- **Data reengineering**: clean-up and restructure system data.

Refactoring is the process of making improvements to a program to **slow down degradation through change**. You can think of refactoring as **'preventative maintenance'** that reduces the problems of future change. Refactoring involves modifying a program to improve its structure, reduce its complexity or make it easier to understand. When you refactor a program, you should not add functionality but rather concentrate on program improvement. Reengineering takes place after a system has been maintained for some time and maintenance costs are increasing. You use automated tools to process and reengineer a legacy system to create a new system that is more maintainable. Refactoring is a **continuous process of improvement** throughout the development and evolution process. It is intended to avoid the structure and code degradation that increases the costs and difficulties of maintaining a system.

'Bad smells' of code are stereotypical situations in which the code of a program can be improved through refactoring:

- **Duplicate code** or very similar code may be included at different places in a program; it can be removed and implemented as a single method or function that is called as required.
- **Long methods** should be redesigned as a number of shorter methods.
- **Switch (case) statements** often involve duplication, where the switch depends on the type of a value or the switch statements may be scattered around a program. In object-oriented languages, you can often use polymorphism to achieve the same thing.
- **Data clumping** occurs when the same group of data items (fields in classes, parameters in methods) re-occur in several places in a program. These can often be replaced with an object that encapsulates all of the data.
- **Speculative generality** occurs when developers include generality in a program in case it is required in the future. This can often simply be removed.

Legacy system management

Organizations that rely on legacy systems must choose a strategy for evolving these systems. The chosen strategy should depend on the system quality and its business value:

- **Low quality, low business value**: should be scrapped.
- **Low-quality, high-business value**: make an important business contribution but are expensive to maintain. Should be re-engineered or replaced if a suitable system is available.
- **High-quality, low-business value**: replace with COTS, scrap completely, or maintain.
- **High-quality, high business value**: continue in operation using normal system maintenance.